

Cahier des Charges – Projet Augmented Battle Bot (T-ESP-700)

1. Cadre du projet
 - 1.1. Contexte
 - 1.2. Objectifs
 - 1.3. Présentation de l'équipe
 - 1.4. Benchmark
2. Conception graphique
 - 2.1. DA du jeu
 - 2.1.1 Pumabot
 - 2.1.2 KnightBot
 - 2.1.3 SamuraiBot
 - 2.1.4 RoninBot
 - 2.2. Aperçu interface utilisateur
3. Fonctionnalités du projet
 - 3.1. Spécifications fonctionnelles
 - 3.1.1. Menu de démarrage
 - 3.1.2. Hub du jeu
 - 3.1.3. Robot de combat
 - 3.1.4. Gameplay
 - 3.1.5. Réalité augmentée
 - 3.1.6. Backend
 - 3.2. Spécifications techniques
 - 3.2.1. Choix des technologies
 - 3.2.2. Architecture du projet
 - 3.2.3 Architecture de la base de données
4. Fonctionnalités techniques
 - 4.1. Fonctionnalités Jeu mobile
 - 4.1.1. Fonctionnalités techniques du robot de combat
 - 4.1.2. Fonctionnalités techniques du Menu de démarrage
 - 4.1.3. Fonctionnalités techniques du Hub
 - 4.1.4. Fonctionnalités techniques du gameplay
 - 4.1.5. Fonctionnalités techniques réalité augmentée
 - 4.2. Fonctionnalités du backend
 - 4.3. Infrastructure cloud basée sur Microsoft Azure
 - 4.3.1. Introduction
 - 4.3.2. Contexte
 - 4.3.3. Services Azure requis
 - 4.4. Architecture cloud
 - 4.4.1. Vue d'ensemble
 - 4.4.2. Détails des composants
 - 4.4.2.1. Azure PlayFab
 - 4.4.2.2. Azure Cosmos DB
 - 4.4.2.3. Azure Blob Storage
 - 4.4.2.3. Azure CDN
 - 4.4.2.4. Azure Functions
 - 4.4.2.5. Azure Monitor/Log Analytics
 - 4.4.2.5. Azure Key Vault
 - 4.5. Considérations AR
 - 4.5.1. Contexte des considérations AR
 - 4.5.2. Rôle de Unity AR Foundation
 - 4.5.3. Rôle initial d'Azure Spatial Anchors
 - 4.5.5 Impact du retrait d'Azure Spatial Anchors
 - 4.5.6. Autres choix possibles pour les sessions AR partagées
 - 4.5.7. Recommandation pour le projet
5. Gestion des risques
 - 5.1. Risques techniques
 - 5.2. Stratégies d'atténuation
6. Ressources nécessaires

1. Cadre du projet

1.1. Contexte

Beaucoup d'entre nous durant notre enfance avons une passion pour collectionner des gammes de jouets que nous adorions que ce soit des voitures, des dinosaures, des barbies... C'est à partir de ce postulat que notre groupe a décidé de raviver les rêves d'enfance en proposant d'offrir ces collections dans le creux de ces anciens enfants.

En effet, notre studio de jeux vidéo Kombo's Ship souhaite développer un jeu mobile en réalité augmentée impliquant la collection de robots personnalisables avec la possibilité d'améliorer leur potentiel de combat. Une fois vos robots préparés, engagez vous dans un duel palpitant contre d'autre joueur afin d'asseoir votre supériorité sur les autres et de reproduire des combats mythiques de vos souvenirs d'enfance.

1.2. Objectifs

Ce jeu a pour objectif d'attirer certaines catégories de personnes. Tout d'abord le jeu devrait attirer les grands et petits collectionneurs qui seront sûrement attirés par l'aspect collection et personnalisable de notre jeu. Mais la cible principale sont les personnes qui ont du mal à se lancer dans la collection à cause du manque de moyens ou du manque de temps pour commencer et gérer une collection car notre jeu est un moyen alternatif pour ces personnes qui pourront posséder leur propre collection sans vraiment se soucier des contraintes conventionnelles.

Pour remplir cet objectif, il faut que le jeu soit suffisamment attrayant pour attirer ces catégories de personnes. Donc le jeu doit effectivement proposer un large panel de robots, chacun ayant un design unique et personnalisable le tout brodé avec un lore/histoire qui puisse donner envie de le posséder.

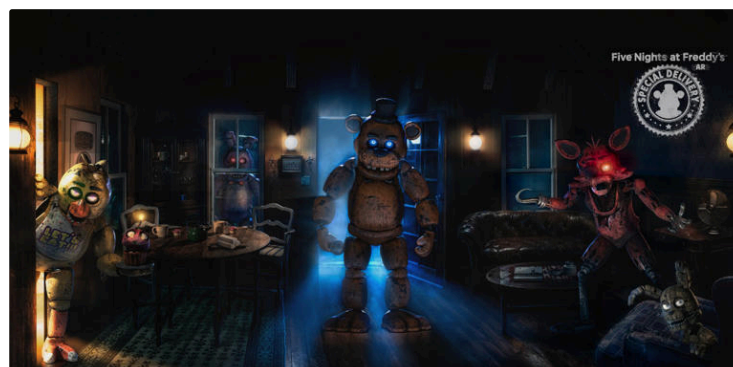
Au-delà de l'aspect collection, il est nécessaire d'amener les futurs joueurs dans une boucle de gameplay qui donnera à ces joueurs une raison de rester sur le jeu. Par conséquent le jeu doit contenir des combats dans lesquels les deux joueurs s'affrontent avec leur robot préféré, combat qui sera formé en ligne via un code de session. Le système de combat de base se doit d'être d'un côté facile à prendre en main mais de l'autre côté, le jeu doit proposer des mécaniques de gameplay demandant de la pratique et qui soient inhérent aux différents robots de telle façon à ce que leur gameplay soient satisfaisants pour les joueurs.

1.3. Présentation de l'équipe



1.4. Benchmark

- Five Nights at Freddy's AR: Special Delivery



Points forts	Points faibles
• Modèle et animation des animatronics • Jumpscare terrifiant • Beaucoup de skin de personnage	• Jeu très buggé • Gameplay assez mauvais • Peu de mise à jour

- Angry Birds AR: Isle of Pigs



Points forts	Points faibles
• Choisie les limites de son niveau • Génération de niveau très détaillé	• Jeu légèrement répétitifs

- Pokemon GO



Points forts	Points faibles
• Concept novateur • Donne envie de sortir pour attraper les pokémons	• Jeu redondant • Marche surtout en zone urbain • Il faut avoir une bonne connexion

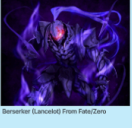
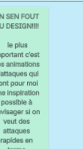
- Permet de visiter des lieux
- Rapprochent les joueurs

2. Conception graphique

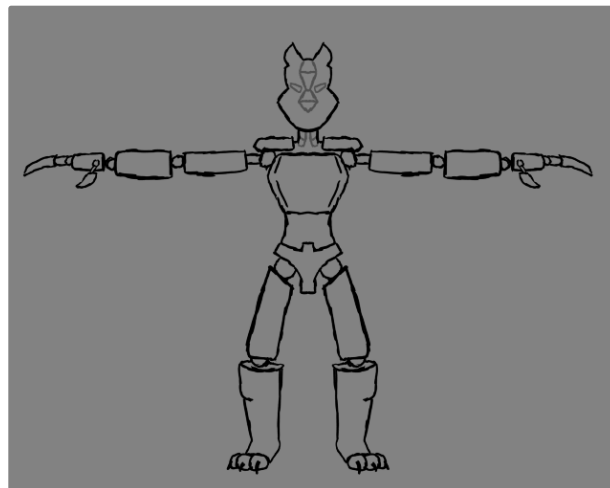
2.1. DA du jeu

La phase de direction artistique passait surtout par une réflexion sur des concepts ou des idées de robots qui pourraient être intéressants à implémenter dans notre jeu. Vous pouvez visualiser ci-dessus des idées de robot nées d'un brainstorming assez intensif.

2.1.1 Pumabot

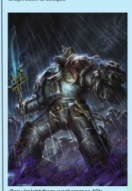
Nom : Blackbot Aspect : - Taille élevée avec des membres assez fins mais longs - Équilibre en peu patte - Graciles mais robuste de griffes - Jambes plus fines par rapport au reste - Buste relativement petit - Robot portant les traits d'un panda/mouton - Jambes ressemblant à des pattes de filin - Ailes minces - Couleur : ... - Principale robe : ... - Accessoires robot : ... - "Yout" vers ou vers le bas ? - S'adresse sur la tête (sans effet de la bouche) Style de combat : Armes : Griffes articulées au niveau des mains et pieds. Attaque : Entre coups de griffes et coup de pieds avec des griffes Capacité de combat : Avantages : - Peut sauter plus de cinq mètres et ses coups sont plus rapides à réajuster. - De plus, il possède une mobilité supérieure aux autres et ses requêtes peuvent un plus grande distance. Défauts : Hélicoptère et PV inférieurs à la normale, points d'attaque plus courts que les autres Attaque spéciale : Saut en vol avec ses griffes ? Utilité : Entre vers l'ennemi à une vitesse impossible à suivre à l'œil nu et envoie plus de coups par seconde sur des adversaires sur l'ennemi	Inspiration artistique  Lil from Prometeia  Benker (Lancelot) from Fate/Zero  Skull  Zoro from One Piece  Zoro from One Piece  Zoro from One Piece ON SEN FOLT DU DESIGN!!! le plus important c'est ses animations d'attaques qui sont pour moi une inspiration possible à intégrer si on veut des attaques rapides en terme d'animation https://youtu.be/Yr9GdC8b0wI https://youtu.be/Yr9GdC8b0wI https://youtu.be/Yr9GdC8b0wI
--	---

Phase de brainstorming pour le Pumabot

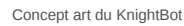


Concept art du Pumabot

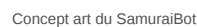
2.1.2 KnightBot

Nom : Robot chevalier ? Aspect : - Taille élevée et corps relativement massif (sans massif qu'un peu) - Équilibre en peu patte - Graciles mais avec une apparence de guerrier de chevalier ? - Buste plus fin par rapport au reste - Robot avec un aspect de chevalier lourd - Robot avec un aspect de chevalier léger - Ailes minces - Couleur : ... - Principale robe : ... - Accessoires robot : ... - "Yout" vers ? - S'adresse sur la tête (sans effet de la bouche) Style de combat : Armes : Lances de chevalier massif et lance - épée Attaque : Entre coups de lance avec le bras de la lance et lance de la lance Capacité de combat : Avantages : - Peut sauter plus de cinq mètres et ses coups sont plus rapides à réajuster. - De plus, il possède une mobilité supérieure aux autres et ses requêtes peuvent un plus grande distance. Défauts : Hélicoptère et PV inférieurs à la normale, points d'attaque plus courts que les autres Attaque spéciale : Charge vers l'ennemi avec la lance de la lance et lance de la lance Utilité : Entre vers l'ennemi à une vitesse impossible à suivre à l'œil nu et envoie plus de coups par seconde sur des adversaires sur l'ennemi	Inspiration artistique  Grey knight from Warhammer 40k  Grey knight from Warhammer 40k  Grey knight from Warhammer 40k  Grey knight from Warhammer 40k  Grey knight from Warhammer 40k  Grey knight from Warhammer 40k ON SEN FOLT DU DESIGN!!! le plus important c'est ses animations d'attaques qui sont pour moi une inspiration possible à intégrer si on veut des attaques rapides en terme d'animation https://youtu.be/Yr9GdC8b0wI https://youtu.be/Yr9GdC8b0wI https://youtu.be/Yr9GdC8b0wI
---	---

Phase de brainstorming pour le KnightBot

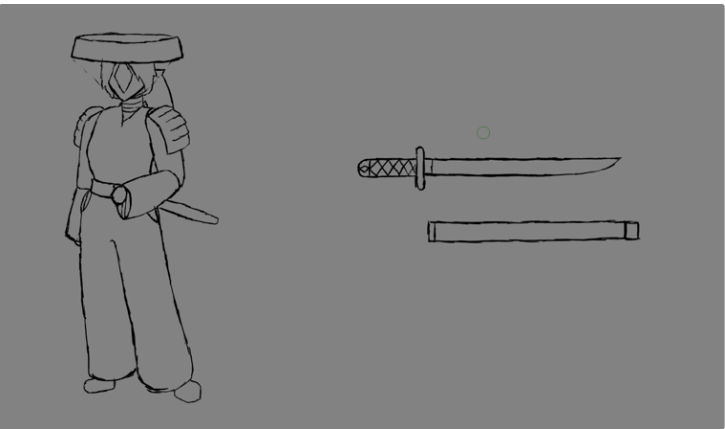
[illegible]

Phase de brainstorming pour le SamuraiBot

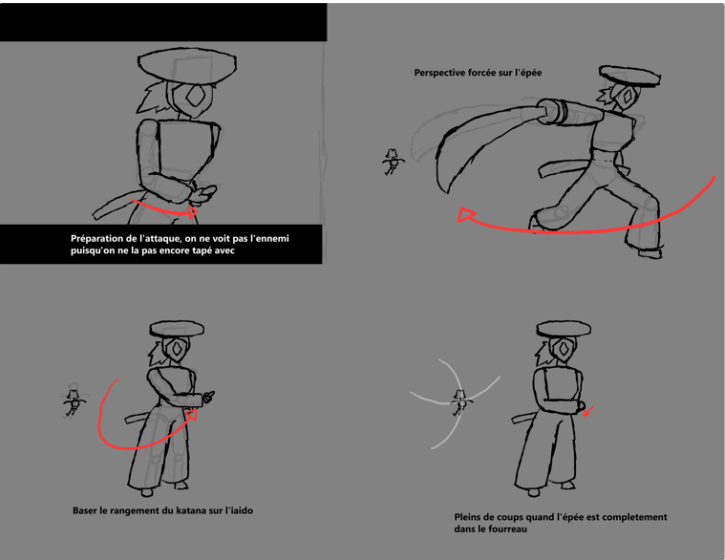


Nom : Robot Ronon avec katana (d'inspiration)	Inspiration artistique	Design qui coche un peu tous les critères si on part sur un robot samurai. Ça a le fait de être plus et pas des vil, en plus ses attaques peuvent être une source d'inspiration pour nos attaques
Aspect : - Taille normale et corps normal - Corps sans ligne qui va gêner le mouvement ? - Équilibre centré ? - Rigidité sans laisser l'aspect déformable ou délabré - Jambes couverts par sa tenue (différent du nôbi) - Casq ? - Couleurs ? - Visuel ? - Jumeau ? - "Real" design ? - Célébrité ? Style de combat : Armes, posture Enchaînement de coups rapides et plus simple et attaque de type quick draw (3 flèches ou ressemblage)	Myamoto Iori From Fate Samurai Remnant Kimbiri Hicira From Karamin le vagabond Ochiai Zoro From Fate/Grand Order Si on veut un style de combat à base de quick-draw, bah il suffit de regarder n'importe quelle œuvre de fiction ou même rien qui montre des combat de samurai. (Donc il faut choisir le sujet)	Weissmoss from Shingeki no Yosei Design qui coche un peu tous les critères si on part sur un robot samurai. Ça a le fait de être plus et pas des vil, en plus ses attaques peuvent être une source d'inspiration pour nos attaques
Capacité de combat : Avantages : - Peut charger sa compétence pour infliger plus de dommages. Plus chaque compétence et devient moins le contre-attaque, autant on combat et augmente sa vitesse d'attaque. Au contraire de son combat, inflige une attaque au Dojô qui lui permet d'être plus de dégâts. De plus sa compétence lui donne des dégâts énormes. Inconvénients : - (Sans déguiser et en raison d'être dans son combat à la normale sans une course. Les compétences sans être chargés ne font pas plus de dommages.	Rolden from Metal Gear Solid Rising Vergil Sparda from Devil May Cry C'est ça fait plus qu'on meche au robot mais on peut apprendre cette compétence pour le design de notre robot Si on veut des attaques très flashy et spectaculaires. On peut emprunter des animations d'attaques	Si on veut des attaques très flashy et spectaculaires. On peut emprunter des animations d'attaques Il faut décider la forme que l'arme va prendre https://twitter.com/atomarobots/status/1044444444444444444 type-de-sabre-ja-pourra
Attaque spéciale : Non chargé - coup à la verticale Chargé - charge avec katana, puis le déplace pour faire un coup horizontal de 90 degrés, plus elle charge la compétence, plus elle fait des dégâts		
Utilité : Range son katana, puis force vers l'arrière pour ensuite décaler son katana et donner une série de coup (plutôt coupe phase, l'acier devient noir et on entend des coups de métal. C'est le robot qui s'arrête dans 3 secondes mais en regardant son katana et des coups de métal apparemment pour traverser le prochain ennemi.		

Phase de brainstorming pour le RoninBot



Concept art du RoninBot



Conception d'une animation d'attaque du RoninBot

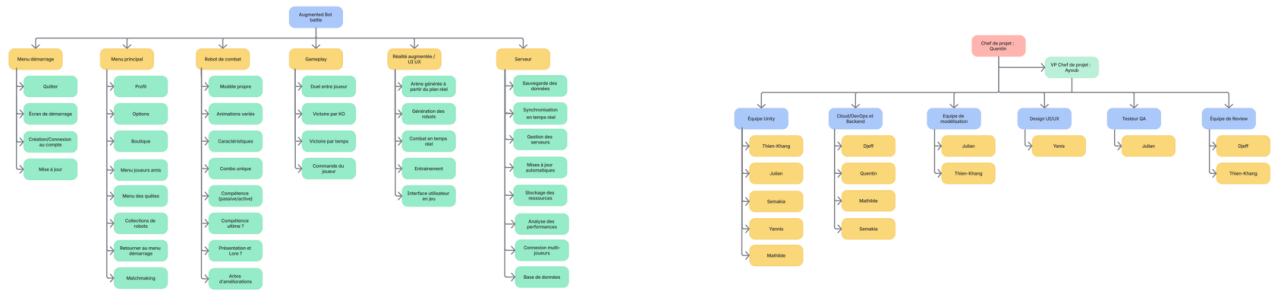
2.2. Aperçu interface utilisateur

Vous consulter le figma de notre jeu avec le lien ci-dessous

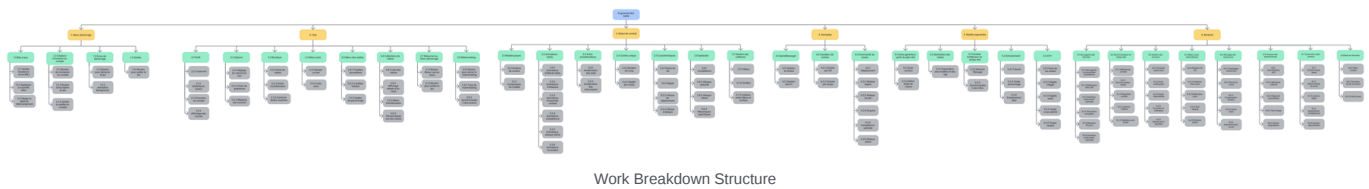
<https://www.figma.com/proto/hcYOIEVsgNOIXe1xWEyzX2/Untitled?node-id=19-4&p=f&t=fIHpk1gZXmZinIXx-0&scaling=min-zoom&content-scaling=fixed&page-id=0%3A1&starting-point-node-id=19%3A4>

3. Fonctionnalités du projet

3.1. Spécifications fonctionnelles



Product Breakdown Structure



NB Pour consulter en détail le PBS et WBS, il est préférable de directement aller sur notre Espace Confluence.

3.1.1. Menu de démarrage

Le menu de démarrage sera la 1^{er} interface que va rencontrer notre joueur au lancement de notre jeu. Sur l'écran de démarrage, le joueur pourra accéder au prochain menu et initialiser le jeu en appuyant sur n'importe quelle touche. De plus, le joueur aura la possibilité de créer ou se connecter à son compte sur l'écran de démarrage, compte qui sera nécessaire pour accéder à notre jeu. Enfin un bouton sera disponible pour quitter le jeu.

3.1.2. Hub du jeu

Le menu principal du jeu va comporter ces différents éléments la plupart accessible via des boutons:

- Profil : Sur cette interface, le joueur pourra visualiser les informations joueurs en plus de ses statistiques montrant ses performances dans le jeu. Le joueur pourra voir son code ami qu'il pourra partager pour que d'autre joueur puisse faire une demande d'ami. Par ailleurs, le joueur pourra observer tous les succès qu'il aura débloqué et ceux à débloquer.
- Options : Dans ce menu, le joueur pourra paramétrer son jeu tel que le volume du son, la qualité graphique du jeu, le mapping des touches..etc.
- Boutique: Le joueur pourra acheter un robot parmi la liste proposée par la boutique en plus des améliorations associées à ces derniers. Il y aura la possibilité de se faire rembourser son/ses achats.
- Menu amis : Grâce à ce menu, le joueur pourra ajouter un ami via le code ami et ainsi l'inviter à jouer ou à discuter maintenant qu'il fait partie de sa liste d'amis. De plus, le joueur pourra bloquer d'éventuels joueurs qui se montreront désagréable.
- Menu des quêtes : le jeu proposera aux joueurs des quêtes qui permettra aux joueurs de gagner des ressources afin d'acheter davantage de robots. Les quêtes seront journalières et hebdomadaires en générales mais il existera des quêtes liés aux personnages qui débloquent des récompenses uniques.
- Collections de robots : le joueur visualisera ses robots possédés dans cette interface. Il pourra notamment voir les détails de ses robots tels que ses statistiques, ses pouvoirs ou son histoire. Il pourra aussi améliorer les robots dans le menu d'amélioration et pourra personnaliser l'apparence de ses robots.
- Système de lobby: Avant de lancer son combat, le joueur va devoir d'abord soit hébergé une partie et invité un joueur avec le code de son lobby, soit il rejoint à un joueur avec le code du lobby qu'on lui a transmis
- Le joueur pourra soit retourner sur le menu de démarrage, soit quitter directement le jeu.

3.1.3. Robot de combat

Nos robots de combats vont être composé de ces éléments :

- Modèle et Animation : Chaque Robot auront un modèle unique avec un schéma de couleur et une forme unique. De plus les robots auront tous une animation unique en fonction du contexte comme par exemple une animation d'attaque, une animation d'attente...
- Stats et style de jeu : Chaque auront le kit suivant
- des stats différentes comme les pv, les dégâts, la vitesse de déplacement, la vitesse d'attaque et une résistance.

De plus, ils auront une série de combos et des compétences propre à eux qui définissent leur style de jeu.

- Pour leur ajouter plus d'attachement, il faut définir une personnalité, une origine et une histoire qui leur donne plus de profondeur et ainsi les joueurs peuvent éprouver plus d'attachement envers eux.

3.1.4. Gameplay

Pour combattre les joueurs pourront déplacer leur robot, donner des coups à leur ennemi avec des attaques légères ou lourdes. De plus, le joueur pourra esquiver les coups avec son robot. Les joueurs auront aussi accès à des compétences et des attaques ultimes aux effets variés en fonction du robot choisi pour le combat.

Durant le combat, un joueur gagne s'il met KO son adversaire ou s'il a le plus de pv à la fin du temps.

3.1.5. Réalité augmentée

Avant le début du combat, une arène sera générée à partir du plan réel en délimitant les limites de l'arène et en générant les obstacles potentiels sur le terrain. Après le terrain, les robots sont ensuite générés après cela, le combat peut commencer. Il faudra bien penser à tester la connexion entre les joueurs pour éviter toute désynchronisation et vérifier que le ping ne soit pas trop élevé.

Le joueur aura accès à un mode entraînement dans lequel il pourra apprendre les mécaniques du jeu ou maîtriser des robots spécifiques tout en lisant un guide sur le personnage choisi.

Dans l'ATH, on retrouve des informations concernant les pv restants du robot, les infligés par le robot, les dégâts subis lors que l'impact d'un coup de l'ennemi, la jauge de coup spéciale et le temps restant avant la fin du combat.

3.1.6. Backend

Une infrastructure cloud sera mise en place afin d'une part pouvoir stocker les informations des joueurs dans une base de données mais aussi de pouvoir afficher leur classement, les robots qu'ils possèdent, gérer le multijoueur en réalité augmentée et notamment la synchronisation entre joueurs. De plus, il sera possible de pouvoir analyser les performances du jeu tel que les revenus du jeu, le nombre de joueurs connecté par jour, l'activité des joueurs...

3.2. Spécifications techniques

3.2.1. Choix des technologies

• .NET

.NET est l'infrastructure sur laquelle tourne le moteur de jeu Unity avec lequel le jeu est réalisé, les deux technologies utilisent le langage C#. Le framework [ASP.NET](#) permet de développer des API afin de communiquer avec des clients et des bases de données. Les fonctionnalités de l'API sont donc de pouvoir faire le lien avec:

- la base de donnée sur laquelle seront stockées les informations des joueurs,
- l'application mobile afin que ces derniers puissent accéder aux fonctionnalités multijoueur

• Unity

Unity est un moteur de jeu multiplateforme qui permet de créer des jeux divers et variés en 2D ou 3D. De plus, Unity possède une bibliothèque de ressources (assets) afin de faciliter le développement de jeu.

Le framework AR Fondation d'Unity permet de créer des applications en Réalité augmentée. L'intérêt principal de ce framework est de pouvoir utiliser les mêmes fonctionnalités AR que dans ARKit et ARCore directement dans un projet Unity évitant donc de recréer un projet AR pour la version Android et IOS.

• ARKit et ARCore

ARKit et ARCore sont des SDK (kits de développement) offrant de nombreuses fonctions dédiées à la création d'applications AR. Les fonctions principaux des SDK sont les suivantes :

- Détections de surfaces
- Ancrages dans l'espace réel
- Suivie de mouvement et positionnement dans l'espace AR
- Occlusion
- Fonctionnalités multi-joueurs (partage d'une même session AR entre plusieurs joueurs)
- Stockages des Ancrages dans un cloud

• Serveur Azure

Voir la partie Architecture et infrastructure cloud

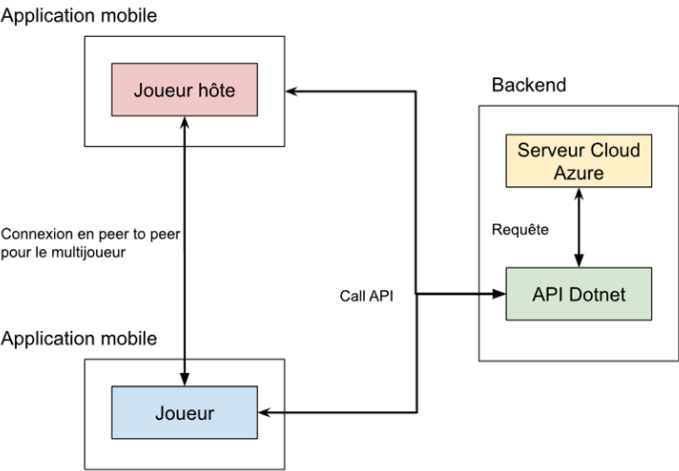
• PostgreSQL

PostgreSQL est un système de gestion de base de données relationnelle open source et facile d'implémentation avec de nombreuses technologies. C'est un outil très répandu dans de nombreux projet qui possède également des fonctionnalités afin de stocker des objets aux format JSON

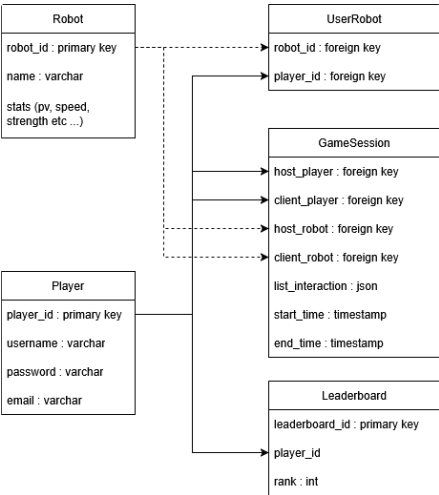
• Blender

Blender est une solution gratuite et open source qui permet de faire de la modélisation 3D et de l’animation. Blender a pour avantage de réunir tous les outils nécessaires pour la modélisation et l’animation tel que le sculpting et le rigging. De plus Blender est compatible aux jeux vidéos créés sur Unity et l’export vers ce moteur de jeu est très facile à faire. Aussi, il existe énormément de d’add-ons et extensions pour améliorer blender. De plus, il existe une énorme communauté qui propose de nombreux tutoriels gratuits pour les non initiés.

3.2.2. Architecture du projet



3.2.3 Architecture de la base de données



4. Fonctionnalités techniques

4.1. Fonctionnalités Jeu mobile

4.1.1. Fonctionnalités techniques du robot de combat

Nom de la fonctionnalité	Classe robot
Description de la fonctionnalité	Le robot possède des stats telles que les points de vie, dégâts, vitesse de déplacement et vitesse d’attaque.

Comportement de la fonctionnalité	La classe robot sera une classe abstract qui aura dans son constructeur les propriétés suivantes : • Hp: int • Damage: int • MoveSpeed : float • AttackSpeed : float NB : En plus des getters et des setters, cette classe aura les méthodes suivantes : • Move • LightAttack • HeavyAttack • SpecialAttack
Contrainte technique	
Critère de validation	• Toutes les propriétés ont été implémentés avec les bon types • Tous les getters et setters ont été implémenté • Les autres méthodes ont été défini

Nom de la fonctionnalité	Mouvement
Description de la fonctionnalité	Méthode qui permet au robot de pouvoir se déplace sur une surface
Comportement de la fonctionnalité	Concrètement lorsque le joueur appuiera sur le bouton pour bouger le robot, sa position sera modifié tant que le joueur pressera le bouton pour bouger.
Contrainte technique	Vector3
Critère de validation	• Lorsque le joueur bougera le robot en pressant un bouton, le modèle du robot a bien changé de position et est dans la bonne direction

Nom de la fonctionnalité	Esquive
Description de la fonctionnalité	Méthode qui permet au robot de pouvoir effectuer un mouvement rapide sur une surface

Comportement de la fonctionnalité	Concrètement lorsque le joueur appuiera sur le bouton pour bouger le robot, sa position sera modifiée d'une valeur fixe dans la direction que le joueur aura choisi.
Contrainte technique	Vector3
Critère de validation	Lorsque le joueur bougera le robot en pressant un bouton, le modèle du robot a bien changé de position et est dans la bonne direction.

Nom de la fonctionnalité	Attaque lourde
Description de la fonctionnalité	Le robot donne un coup lourd, infligeant des dégâts à son ennemi si le coup touche.
Comportement de la fonctionnalité	Lorsque le joueur appuie sur le bouton pour donner un coup lourd, le robot lance son animation pour donner un coup lourd et le jeu vérifie si la hitbox du coup touche la hurtbox de son ennemi. Si le coup touche, l'ennemi prend des dégâts sinon il ne se passe rien. NB : le joueur ne pourra pas prolonger un combo après l'utilisation d'une attaque lourde
Contrainte technique	Animation, Timing, Collision
Critère de validation	- L'animation du coup lourd est bien exécuté - Le coup touche bien réellement l'ennemi - Les dégâts et ses effets éventuels sont bien appliqués

Nom de la fonctionnalité	Attaque légère
Description de la fonctionnalité	Le robot donne un coup léger, infligeant des dégâts à son ennemi si le coup touche.
Comportement de la fonctionnalité	Lorsque le joueur appuie sur le bouton pour donner un coup léger, le robot lance son animation pour donner un coup léger et le jeu vérifie si la hitbox du coup touche la hurtbox de son ennemi. Si le coup touche, l'ennemi prend des dégâts sinon il ne se passe rien.

	NB : le joueur pourra prolonger des combos avec des attaques légères
Contrainte technique	Animation, Timing, Collision
Critère de validation	• L'animation du coup léger est bien exécuté • Le coup touche bien réellement l'ennemi • Les dégâts et ses effets éventuels sont bien appliqués

Nom de la fonctionnalité	Compétence (abandonnée)
Description de la fonctionnalité	Le robot utilise une compétence qui procure divers effets
Comportement de la fonctionnalité	Lorsque le joueur appuie sur le bouton pour utiliser la compétence, le robot lance son animation pour utiliser la compétence et le jeu applique les effets en fonction du robot joué. • Soit la compétence modifie les stats du robot ou/et et de son adversaire • Soit le robot attaque son ennemi et applique divers effets et dégâts si la compétence touche la cible (cette attaque est considéré comme un attaque spéciale) NB : le joueur ne pourra pas enchaîner un combo après l'utilisation d'une compétence
Contrainte technique	Animation, Timing, Collision
Critère de validation	• L'animation de la compétence est bien exécuté • Les effets et dégâts éventuels sont bien appliqués • La compétence est en récupération après son utilisation

Nom de la fonctionnalité	Attaque spéciale
Description de la fonctionnalité	Le robot consomme une partie de son énergie pour effectuer une attaque puissante

Comportement de la fonctionnalité	Lorsque le joueur appuie sur le bouton pour utiliser l'attaque spéciale, si le robot a assez d'énergie, Il lance son animation pour utiliser l'attaque spéciale Le jeu vérifie si la hitbox du coup touche la hurtbox de son ennemi. Si le coup touche, l'ennemi prend des dégâts sinon il ne se passe rien. NB : le joueur ne pourra pas enchaîner un combo après l'utilisation d'une attaque spéciale
Contrainte technique	Animation, Timing, Collision
Critère de validation	• L'animation de l'attaque spéciale est bien exécuté • Les dégâts et les effets éventuels sont bien appliqués • Il faut que le joueur ait assez d'énergie pour effectuer l'attaque spéciale

Nom de la fonctionnalité	Attaque ultime (abandonnée)
Description de la fonctionnalité	Le robot consomme toute son énergie pour effectuer une attaque dévastatrice
Comportement de la fonctionnalité	Lorsque le joueur appuie sur le bouton pour utiliser l'attaque spéciale, si le robot a assez d'énergie, Il lance son animation pour utiliser l'attaque ultime Le jeu vérifie si la hitbox du coup touche la hurtbox de son ennemi. Si le coup touche, l'ennemi prend des dégâts sinon il ne se passe rien. NB : le joueur ne pourra pas enchaîner un combo après l'utilisation d'une attaque ultime
Contrainte technique	Animation, Timing, Collision
Critère de validation	• L'animation de l'attaque ultime est bien exécuté • Les dégâts et les effets éventuels sont bien appliqués • Il faut que le joueur ait assez d'énergie pour effectuer l'attaque

	ultime
--	--------

Nom de la fonctionnalité	Gestion des combos
Description de la fonctionnalité	Le joueur peut enchaîner plusieurs coups légers en fonction du robot choisi
Comportement de la fonctionnalité	Pour chaque attaque légère donnée par le joueur, si le délai entre chaque coup est suffisamment court, alors une animation différente sera lancée par le joueur lors du prochain coup et ce coup aura une hitbox différente.
Contrainte technique	Animation, Timing, Collision
Critère de validation	• Durant un combo, chaque coup donné a une animation différente • Durant un combo, chaque coup donné inflige des dégâts différents • Si il y a une pause trop longue entre deux coups, le combo est réinitialisé

Nom de la fonctionnalité	Gestion des collisions
Description de la fonctionnalité	Le robot possède une hitbox et une hurtbox
Comportement de la fonctionnalité	Lors de la création du gameObject représentant le robot, il faut qu'elle possède deux groupes de colliders : • Le premier pour la hurtbox qui servira à détecter si il y a des collisions avec d'autre gameObject • Le deuxième sera pour la hitbox des coups donnés par le robot et son ennemi et ce coup inflige des dégâts si il y a une collision entre la hitbox et la hurtbox
Contrainte technique	Collision
Critère de validation	• Le robot possède bien une hitbox et hurtbox • Une collision est détecté si le robot entre en contact avec un autre gameObject • Il y a bien une collision qui est détecté si la hitbox et la hurtbox

	se touchent
--	-------------

Nom de la fonctionnalité	Amélioration des stats (abandonnée)
Description de la fonctionnalité	En fonction des achats du joueur sur le robot, des améliorations de stats sera appliqués sur le robot
Comportement de la fonctionnalité	En reprenant la classe robot, il faudra ajouter des propriétés qui modifieront les stats finaux du robot. Il faut vérifier au préalable si le joueur a bien acheté l'amélioration correspondant avant d'appliquer l'amélioration
Contrainte technique	classe robot, améliorations
Critère de validation	• L'amélioration des stats a bien été appliqué avant le combat • L'amélioration a bien été acheté avant l'application des stats

Nom de la fonctionnalité	Amélioration des mécaniques (abandonnée)
Description de la fonctionnalité	En fonction des achats du joueur sur le robot, des améliorations des mécaniques sera appliqués sur le robot
Comportement de la fonctionnalité	En reprenant la classe robot, il faudra ajouter des propriétés qui amélioreront les mécaniques du robot. Il faut vérifier au préalable si le joueur a bien acheté l'amélioration correspondant avant d'appliquer l'amélioration
Contrainte technique	classe robot, améliorations
Critère de validation	• L'amélioration des mécaniques a bien été appliqué avant le combat • L'amélioration a bien été acheté avant l'application des amélioration

Nom de la fonctionnalité	Lock on
Description de la fonctionnalité	Fonctionnalité qui permet au robot de fixe automatiquement son adversaire
Comportement de la fonctionnalité	Le robot va automatiquement faire des rotations pour fixer son

	adversaire. Sachant que lorsque le robot sera dans une action, le robot ne va plus automatiquement fixer son adversaire
Contrainte technique	Vector3, Quaternion
Critère de validation	- Le robot fixe bien son adversaire automatiquement - Lors d'une action, le robot ne fixera plus automatiquement son adversaire

4.1.2. Fonctionnalités techniques du Menu de démarrage

Nom de la fonctionnalité	Interface menu de démarrage
Description de la fonctionnalité	Premier Interface qui permet à l'utilisateur de lancer le jeu
Comportement de la fonctionnalité	Lorsque le joueur va lancer l'application, il va d'abord atterrir sur le menu de démarrage avec un joli background et différents boutons aux fonctionnalités variées
Contrainte technique	Canvas, panel, text, image, button
Critère de validation	Lorsqu'on lance l'application, le joueur est bien redirigé sur le menu de démarrage et avec les différents différents boutons initialisés

Nom de la fonctionnalité	Bouton pour démarrer le jeu
Description de la fonctionnalité	Ce bouton permet de rediriger le joueur depuis le menu de démarrage vers le menu principal
Comportement de la fonctionnalité	Une fois sur le menu de démarrage, le joueur lancera véritablement le jeu depuis le bouton démarrer, ce qui lancera un chargement et le joueur sera redirigé vers le menu
Contrainte technique	Canvas, panel, button, text
Critère de validation	Lorsqu'on appuie sur le bouton, le jeu est initialisé et le joueur est redirigé vers le menu principal

Nom de la fonctionnalité	Bouton d'inscription au jeu
--------------------------	-----------------------------

Description de la fonctionnalité	Ce bouton permet au joueur de se créer un compte pour le jeu
Comportement de la fonctionnalité	Lorsque le joueur appuiera sur le bouton, un formulaire va s'ouvrir dans lequel le joueur devra le remplir avec les informations suivants : • Nom • Prénom • Nom d'utilisateur • Numéro de téléphone • Date de naissance • Lieu d'habitation • Email • Mot de passe Une fois que ces informations ont été validé, le compte sera créé et stocké dans la base de données et par la suite, le joueur sera connecté avec ce compte et pourra ainsi accéder au reste du jeu NB : Pour accéder au contenu du jeu, il est nécessaire d'être connecté à un compte
Contrainte technique	Canvas, panel, button, text, input field Communication avec la base de données
Critère de validation	• Lorsqu'on appuie sur le bouton, un formulaire s'affiche avec tous les champs nécessaires à remplir • Après avoir validé les informations données, une requête sera envoyée au serveur pour créer le compte et le stocker dans la base de données • Après la création du compte, le joueur est connecté à son nouveau compte et ainsi il peut accéder au menu principal et au reste du jeu.

Nom de la fonctionnalité	Bouton de connexion au jeu
--------------------------	----------------------------

Description de la fonctionnalité	Ce bouton permet au joueur de se connecter à son compte pour le jeu
Comportement de la fonctionnalité	Lorsque le joueur appuiera sur le bouton, un formulaire va s'ouvrir dans lequel le joueur devra le remplir avec les informations suivants : • Email • Mot de passe Une fois que ces informations ont été validé, une requête sera envoyé pour vérifier si le compte existe bien dans la base de données et si c'est le cas, le joueur sera connecté avec ce compte et pourra ainsi accéder au reste du jeu NB : Pour accéder au contenu du jeu, il est nécessaire d'être connecté à un compte
Contrainte technique	Canvas, panel, button, text, input field Communication avec la base de données
Critère de validation	• Lorsqu'on appuie sur le bouton, un formulaire s'affiche avec tous les champs nécessaire à remplir • Après avoir validé les informations donné, une requête sera envoyé au serveur pour vérifier si le compte existe bien dans la base de données • Après la création du compte, le joueur est connecté à son nouveau compte et ainsi il peut accéder au menu principal et au reste du jeu.

Nom de la fonctionnalité	Mise à jour du jeu (abandonnée)
Description de la fonctionnalité	Vérifie si il y a une mise à jour disponible et le télécharge
Comportement de la fonctionnalité	Lorsque le joueur appuiera sur le bouton pour démarrer le jeu, il y aura une vérification de mise à jour avant et si une mise à jour

	disponible, le jeu ira télécharger la mise à jour (en téléchargeant les différents packages nécessaires) à la fin de la mise à jour, le joueur pourra accéder au reste du jeu NB : il sera possible de régler le débit de téléchargement
Contrainte technique	
Critère de validation	• La vérification de mise à jour se lance bien avant que le joueur accède au menu principal • Le téléchargement est bien lancé pour mettre à jour le jeu • Le jeu se lance correctement après la mise à jour

Nom de la fonctionnalité	Bouton pour quitter le jeu
Description de la fonctionnalité	Ce bouton permet au joueur quitter le jeu
Comportement de la fonctionnalité	Lorsque le joueur appuiera sur le bouton, le joueur quittera le jeu
Contrainte technique	Canvas, panel, button, text,
Critère de validation	• Le jeu s'éteint correctement

4.1.3. Fonctionnalités techniques du Hub

Nom de la fonctionnalité	Interface hub principal
Description de la fonctionnalité	Hub principal qui permet de se rediriger vers d'autres menus
Comportement de la fonctionnalité	Lorsque le joueur arrivera dans le hub principal, il verra certaines informations tel que son nom, son niveau, son argent... Mais surtout, le joueur pourra accéder aux autres menus via des boutons tel que : • Son profil • Les options du jeu • La boutique • Sa liste d'ami • Le menu des quêtes • Sa collections de robots • Le lancement de matchmaking
Contrainte technique	Canvas, panel, button, text,
Critère de validation	• Le menu principal s'affiche bien • Le hub contient tous les boutons permettant d'accéder à autres menus

Nom de la fonctionnalité	Interface profil du joueur
Description de la fonctionnalité	Interface qui affiche toutes les informations du joueur
Comportement de la fonctionnalité	Dans cette interface, le joueur pourra voir toutes ses informations obtenues grâce à une requête vers la base de données en passant par le serveur. Voici les informations qui seront affichés : • Ses informations utilisateurs • Son code ami • Ses statistiques (performance de combat) • Ses succès
Contrainte technique	Canvas, panel, button, text, Communication avec la base de données
Critère de validation	• Lorsque le joueur accède à son profil, le jeu parvient à obtenir les informations du joueur via une requête à la base de données en passant par le serveur

Nom de la fonctionnalité	Interface des options
Description de la fonctionnalité	Interface qui permet au joueur de modifier les options du jeu
Comportement de la fonctionnalité	Dans cette interface, le joueur modifier les options suivants : • Volume du son • Choix des graphismes • Accessibilité
Contrainte technique	Canvas, panel, button, text,
Critère de validation	• Le joueur accède au menu options sans problème et les différentes options sont bien affichés • Les modifications des options sont bien appliqués • Possibilité de restaurer les options par défauts

Nom de la fonctionnalité	Interface de la boutique
Description de la fonctionnalité	Interface qui permet au joueur d'acheter des robots, des améliorations ou/et des boites mystères
Comportement de la fonctionnalité	Dans cette interface, le joueur aura accès à un certain nombre de produits qu'il pourra acheter. la boutique affichera ces listes de produits : • La liste des robots disponibles • La liste des améliorations Il sera possible d'appliquer un filtre pour trier les robots et améliorations en fonction du critère De plus, le joueur pourra ouvrir des boîtes mystères. Pour avoir ces listes de produits, le jeu doit faire une requête à la base de données pour obtenir les produits voulus NB : Pour plus de détails sur les listes et boîtes mystère, vous pouvez regarder leur fonctionnalités dédiés
Contrainte technique	Canvas, panel, button, text, image
Critère de validation	• Le joueur parvient à obtenir les produits via la requête vers la base de données • les listes de produits sont bien affichés dans la boutique • Les listes peuvent être filtré

Nom de la fonctionnalité	Achat d'un robot
Description de la fonctionnalité	Le joueur peut acheter un robot avec son argent virtuel
Comportement de la fonctionnalité	Lorsque le joueur achète un robot, son argent sera déduite et le robot acheté sera enregistré sur le compte du joueur afin qu'il puisse l'utiliser pour les combats

	Au moment de l'achat, le jeu fera une requête à la base de données pour enregistrer le robot sur le compte du joueur et le compte sera mise à jour
Contrainte technique	Canvas, panel, button, text, Communication avec la base de données
Critère de validation	- Après l'achat, le robot est bien enregistré sur le compte du joueur - L'argent du joueur est bien déduit après l'achat - Le joueur ne peut pas acheter plus d'une fois le même robot

Nom de la fonctionnalité	Achat d'une amélioration (abandonnée)
Description de la fonctionnalité	Le joueur peut acheter une amélioration avec son argent virtuel
Comportement de la fonctionnalité	Lorsque le joueur achètera une amélioration, son argent sera déduit et l'amélioration achetée sera enregistrée sur le compte. L'amélioration s'appliquera sur le robot en question pour tous les combats Au moment de l'achat, le jeu fera une requête à la base de données pour enregistrer l'amélioration sur le compte du joueur et le compte sera mise à jour
Contrainte technique	Canvas, panel, button, text, Communication avec la base de données
Critère de validation	- Après l'achat, l'amélioration est bien enregistré sur le compte du joueur - L'argent du joueur est bien déduit après l'achat - Le joueur ne peut pas acheter plus d'une fois la même amélioration

Nom de la fonctionnalité	Achat d'une boîte mystère (abandonnée)
Description de la fonctionnalité	Le joueur peut payer une boîte mystère pour avoir une récompense aléatoire
Comportement de la fonctionnalité	Lorsque le joueur achètera une amélioration, son argent sera déduit et un robot aléatoire sera ajouté à la collection du joueur afin qu'il puisse l'utiliser pour les combats Au moment de l'achat, le jeu fera une requête à la base de données pour enregistrer le robot sur le compte du joueur et le compte sera mise à jour
Contrainte technique	Canvas, panel, button, text, Communication avec la base de données
Critère de validation	• Après l'achat, Un robot aléatoire est enregistré sur le compte du joueur • L'argent du joueur est bien déduit après l'achat • La probabilité d'avoir un robot est bien équilibrée

Nom de la fonctionnalité	Interface menu ami (abandonnée)
Description de la fonctionnalité	Interface qui affiche la liste d'ami du joueur
Comportement de la fonctionnalité	Lorsque le joueur arrive sur l'interface menu ami, une liste d'amis sera affichée après avoir interagi avec la base de données pour et il aura la possibilité d'ajouter des amis à partir d'un code ami ou alors de bloquer un joueur. NB : Pour plus détails sur ajouter un ami et bloquer un joueur, il faut regarder leur fonctionnalités dédiés
Contrainte technique	Canvas, panel, button, text, Communication avec la base de données

Critère de validation	• Le menu des joueurs est bien affiché • Le jeu arrive à obtenir la liste des amis du joueur et à les afficher • Les boutons pour ajouter un ami et bloquer un joueur sont bien présents
-----------------------	--

Nom de la fonctionnalité	Ajouter un ami (abandonnée)
Description de la fonctionnalité	Le joueur peut envoyer une demande d'ami à un autre joueur s'il connaît son code ami
Comportement de la fonctionnalité	Lorsque le joueur appuiera sur le bouton ami, il devra taper le code ami dans un champ et une fois faite, le jeu interagira avec la base de données en passant par le serveur pour vérifier si le code ami et par extension le joueur existe. Si c'est le cas, une demande sera envoyée au joueur. Lorsque l'autre joueur a reçu la demande, il a le choix d'accepter ou de refuser la demande d'ami. S'il accepte, les deux joueurs seront ami et leurs listes d'ami respectives seront mise à jour via une requête vers la base de données
Contrainte technique	Canvas, panel, button, text, Communication avec la base de données
Critère de validation	• Lorsque le joueur appuie sur le bouton ajouter un ami, un formulaire apparaît bien • Après avoir mis le code ami, une requête est envoyé à la base de données en passant par le serveur pour vérifier si le code ami et par extension le joueur existe bien dans la base de données • L'autre joueur obtient bien une demande d'ami • Si l'autre joueur accepte la demande d'ami, les deux joueurs

	ont bien leur liste d'ami mise à jour
--	---------------------------------------

Nom de la fonctionnalité	Interface menu des quêtes
Description de la fonctionnalité	Cette interface affiche toute les quêtes du joueur
Comportement de la fonctionnalité	Dans ce menu, le joueur verra toutes les quêtes qu'il doit effectuer. Les quêtes sont divisés de la manières suivantes : • Les quêtes journalières • Les quêtes hebdomadaires • Les quêtes de personnage Chacune de ces quêtes donnent des récompenses plus ou moins conséquentes en fonction de la difficulté. Les quêtes quotidiennes et hebdomadaires se réinitialisent de façon périodique alors que les quêtes de personnages sont valables qu'une seule fois
Contrainte technique	Canvas, panel, button, text,
Critère de validation	• Les différentes quêtes sont bien affichés dans le menu des quêtes • Les joueurs peuvent récupérer leurs récompenses s'ils ont rempli les conditions nécessaires • Les quêtes se réinitialisent par périodes bien sauf les quêtes de personnage

Nom de la fonctionnalité	Interface collection de robots
Description de la fonctionnalité	Cette interface affiche tous les robots que le joueur possède

Comportement de la fonctionnalité	Le joueur pourra consulter sa collection de robots dans cette interface. Lorsque le joueur sera redirigé vers ce menu. Une requête sera lancée vers la base de données en passant par le serveur dans le but d'obtenir la liste des robots débloqués par le joueur.
Contrainte technique	Canvas, panel, button, text,
Critère de validation	• L'interface collection de robot s'affiche bien • Le jeu parvient de faire une requête à la base de données en passant par le serveur pour obtenir la liste des robots • La liste des robots s'affiche bien dans le menu

Nom de la fonctionnalité	Affichage des détails d'un robot
Description de la fonctionnalité	Lorsque le joueur choisit un robot dans le menu de collection, cela lui affiche les détails du robot.
Comportement de la fonctionnalité	Le joueur pourra consulter les détails d'un robot qu'il possède. lorsqu'il fait cela, un autre interface va s'afficher qui va montrer les informations suivantes : • Le nom du robot • Les stats du robot • Un résumé du robot
Contrainte technique	Canvas, panel, button, text,
Critère de validation	• Les détails du robot sont bien affichés

Nom de la fonctionnalité	Menu d'amélioration (abandonnée)
Description de la fonctionnalité	Lorsque le joueur regarde les détails d'un robot, il peut aussi choisir les améliorations du robot en question
Comportement de la fonctionnalité	Lorsque le joueur choisit une amélioration pour le robot choisi, le robot possédé par le joueur est mis à jour et l'amélioration est active pour le reste du jeu.
Contrainte technique	Canvas, panel, button, text,

	Communication avec la base de données
Critère de validation	• L'amélioration choisie a bien été appliqué au robot

Nom de la fonctionnalité	Personnalisation des robots (abandonnée)
Description de la fonctionnalité	Lorsque le joueur regarde les détails d'un robot, il peut aussi choisir le skin du robot en question
Comportement de la fonctionnalité	Lorsque le joueur choisit un skin pour le robot choisi, le robot possédé par le joueur est mis à jour et le skin est actif pour le reste du jeu jusqu'à que le joueur décide de changer de skin.
Contrainte technique	Canvas, panel, button, text, Communication avec la base de données
Critère de validation	• le skin choisi a bien été appliqué au robot

Nom de la fonctionnalité	Système de lobby
Description de la fonctionnalité	Permet à un joueur d'héberger un lobby et qu'un autre joueur puisse rejoindre le lobby
Comportement de la fonctionnalité	Un joueur va hoster la session et l'autre joueur fait office de client. Le joueur client devra insérer un code que le joueur hôte possède. Les données entre les joueurs s'échangent en peer to peer afin qu'ils soient synchronisés.
Contrainte technique	Canvas, panel, button, text, API Communication avec la base de données
Critère de validation	• le skin choisi a bien été appliqué au robot • Les données des joueurs ont étaient vérifiés

Nom de la fonctionnalité	Retour vers le menu de démarrage
Description de la fonctionnalité	Bouton qui permet au joueur de retourner vers le menu de démarrage
Comportement de la fonctionnalité	Lorsque le joueur appuie sur ce bouton, Le joueur est redirigé vers le menu de démarrage
Contrainte technique	Canvas, panel, button, text, Communication avec la base de données
Critère de validation	Le joueur est bien redirigé vers le menu de démarrage sans problème

Nom de la fonctionnalité	Quitter le jeu
Description de la fonctionnalité	Bouton qui permet au joueur de quitter le jeu
Comportement de la fonctionnalité	Idem
Contrainte technique	Canvas, panel, button, text, Communication avec la base de données
Critère de validation	Le jeu s'éteint correctement lorsque le joueur appuie sur le bouton quitter le jeu

Nom de la fonctionnalité	Bouton joueur
Description de la fonctionnalité	Bouton qui permet au joueur d'héberger un lobby ou de rejoindre un lobby en entrant un code
Comportement de la fonctionnalité	
Contrainte technique	Canvas, panel, button, text, Communication avec la base de données
Critère de validation	Le matchmaking s'est bien lancé lorsque le joueur a appuyé sur le bouton

Nom de la fonctionnalité	Type de Matchmaking
--------------------------	---------------------

Description de la fonctionnalité	Avant de lancer le matchmaking, le joueur pourra paramétrer le matchmaking
Comportement de la fonctionnalité	
Contrainte technique	Canvas, panel, button, text, Communication avec la base de données
Critère de validation	Les paramètres choisis par le joueur sont bien appliqués lors du lancement du matchmaking

4.1.4. Fonctionnalités techniques du gameplay

Nom de la fonctionnalité	Game Manager
Description de la fonctionnalité	Le Game Manager est une classe servant à la gestion logique du jeu notamment durant le combat entre robots
Comportement de la fonctionnalité	Lors d'une instance de combat qui commence, le Game Manager sera initialisé afin de gérer l'arrêt du jeu qui se produira notamment durant ces cas : - Lorsque le temps de départ s'est totalement écoulé - Lorsque un des deux joueurs a été battu
Contrainte technique	Singleton ?
Critère de validation	- Le game manager est bien initialisé - Le temps de combat défile bien - Les PV des joueurs sont modifiés en temps réel - Le jeu s'arrête bien lorsque le temps est écoulé ou alors lorsque un des deux joueurs est battu

Nom de la fonctionnalité	Commandes des joueurs
Description de la fonctionnalité	Pour effectuer des actions, le joueurs devra presser les boutons en question
Comportement de la fonctionnalité	Pour contrôler son robot, le joueur devra presser certains boutons pour effectuer les actions qu'il veut

	sachant qu'il y aura deux types d'input possible pour les contrôles : • Des boutons classiques sur l'écran de smartphone • Un système de glissement de doigts NB : Il est possible d'implémenter les deux système d'input dans le futur mais pour l'instant il faudra faire le choix d'un seul système d'input au début
Contrainte technique	Unity input
Critère de validation	• Lorsque la commande est pressée, l'action associée est bien exécutée

4.1.5. Fonctionnalités techniques réalité augmentée

Nom de la fonctionnalité	Scan de la surface de l'arène
Description de la fonctionnalité	Avant de lancer, le combat, joueurs devront scanner une surface afin de générer l'arène de combat
Comportement de la fonctionnalité	Le joueur entrera dans une session AR dans laquelle il devra utiliser la caméra de son smartphone afin de scanner une surface plate pour générer l'arène de combat en définissant bien les limites de l'arène
Contrainte technique	AR Foundation avec ARKit et ARCore AR Session, AR Session Origin, AR Camera, Plane Manager, AR Raycast Manager
Critère de validation	• Après avoir trouvé un combat, les joueurs sont bien entré dans la sessions AR • La caméra des joueurs est capable de détecter des surfaces plates • Une fois une surface détectée, l'arène est bien généré

Nom de la fonctionnalité	Génération des robots dans la session AR
Description de la fonctionnalité	Après que l'arène soit générée, les robots choisi par les joueurs sont

	généérés à leur tour
Comportement de la fonctionnalité	Une fois l'arène générée, les robots des joueurs doivent être générés sur la surface scannée par le joueur. La génération des robots comprend les modèles des robots ainsi que leur rigs
Contrainte technique	AR Foundation avec ARKit et ARCore AR Session, AR Session Origin, AR Camera, Plane Manager, AR Raycast Manager Prefabs, Rigs
Critère de validation	- Après la création de l'arène, la génération des robots est bien lancée - Lors de la génération des robots, leur modèles et rigs sont bien importés

Nom de la fonctionnalité	Network Manager
Description de la fonctionnalité	Entité qui gère les sessions multijoueur entre les joueurs
Comportement de la fonctionnalité	Le Network manager possède une couche de transport (unity) qui lui permet de gérer les transferts de paquets dans les sessions multijoueurs. Dans nos sessions, il va y avoir un joueur host et un joueur client.
Contrainte technique	- Network object (décors, robots...), - Player network object (sous type joueurs), Network behaviours (script pour les networks object), - Network animator (synchronisation des animations), - Network transform (position et rotation des objets network objet)
Critère de validation	- Le joueur host peut host une partie multijoueurs - Un joueur client peut rejoindre la session multijoueurs - Les deux joueurs peuvent interagir en temps réel (donner

	des coups, modification des états...)
--	---------------------------------------

Nom de la fonctionnalité	Synchronisation des informations
Description de la fonctionnalité	Synchronisation des informations par interpolation client
Comportement de la fonctionnalité	Lorsque le client effectue une action, l'action aura du délai. En parallèle, l'autre client va stocker les infos de l'action dans un buffer. Une fois que le délai est écoulé, le buffer est vidé et l'action est effectuée dans la session multijoueur pour les deux joueurs
Contrainte technique	• Ajouter du délai au RTT(Road Trip Time) entre l'hôte et client • Buffer pour stocker les infos de l'action à venir • Au moment du rendu de l'action, calculer un état interpolé à partir des deux ticks du serveur (infos reçu par le joueur)
Critère de validation	• Lorsque un joueur effectue une action, le joueur aura du délai afin que l'action soit synchronisée pour les deux joueurs

Nom de la fonctionnalité	Tutoriel
Description de la fonctionnalité	Session AR pour que le joueur puisse apprendre les bases du jeu
Comportement de la fonctionnalité	Comme pour une partie normale, le joueur devra d'abord scanner une surface pour lancer le tutoriel Dans cette session AR, de nombreuses explications seront données aux nouveaux joueurs afin qu'ils se familiarisent avec l'interface du jeu, le gameplay et les mécaniques du jeu et aussi les commandes à connaître Durant le tutoriel, le joueur contrôlera un robot par défaut
Contrainte technique	AR Foundation avec ARKit et ARCore

	AR Session, AR Session Origin, AR Camera, Plane Manager, AR Raycast Manager Prefabs, Rigs Canvas, panel, button, text,
Critère de validation	• Après avoir scanné une surface, le tutoriel se lance bien • Le robot est bien généré avec son modèle et rigs • Les explications s'affichent bien

Nom de la fonctionnalité	Guide de personnage
Description de la fonctionnalité	Session Ar qui permet au joueur de pouvoir apprendre à maîtriser un robot en particulier
Comportement de la fonctionnalité	Comme pour une partie normale, le joueur devra d'abord scanner une surface pour lancer le tutoriel Fonctionnement similaire au tutoriel sauf que cette session sera plutôt orienté pour les joueurs ayant déjà maîtrisé les bases du jeu et qui veulent maîtriser le robot de leur choix Durant le tutoriel, le joueur pourra choisir le robot qu'il souhaite
Contrainte technique	AR Foundation avec ARKit et ARCore AR Session, AR Session Origin, AR Camera, Plane Manager, AR Raycast Manager Prefabs, Rigs Canvas, panel, button, text,
Critère de validation	• Après avoir scanné une surface, le tutoriel se lance bien • Le robot est bien généré avec son modèle et rigs • Les explications s'affichent bien

Nom de la fonctionnalité	Entraînement libre
Description de la fonctionnalité	Session Ar qui permet au joueur de s'entraîner librement
Comportement de la fonctionnalité	Comme pour une partie normale, le joueur devra d'abord scanner une surface pour lancer le tutoriel Fonctionnement similaire au tutoriel sauf que cette session sera plutôt orienté pour les joueurs ayant déjà maîtrisé les bases du jeu et qui veulent maîtriser le robot de leur choix Durant cette session, le joueur pourra choisir le robot qu'il souhaite
Contrainte technique	AR Foundation avec ARKit et ARCore AR Session, AR Session Origin, AR Camera, Plane Manager, AR Raycast Manager Prefabs, Rigs Canvas, panel, button, text,
Critère de validation	- Après avoir scanné une surface, le tutoriel se lance bien - Le joueur peut choisir son robot avant de lancer le tutoriel du personnage - Le robot est bien généré avec son modèle et rigs - Les explications s'affichent bien

Nom de la fonctionnalité	ATH
Description de la fonctionnalité	Interface du joueur qui sert à afficher diverses informations pendant le combat
Comportement de la fonctionnalité	Durant le combat, le joueur aura un interface qui lui indiquera les informations suivantes : - Les points de vie du joueur et de son adversaire - Son énergie et celui de l'adversaire

	• le temps restant • les dégâts infligés par le joueur • les dégâts infligés par l'ennemi
Contrainte technique	AR Foundation avec ARKit et ARCore AR Session, AR Session Origin, AR Camera, Plane Manager, AR Raycast Manager Prefabs, Rigs Canvas, panel, button, text,
Critère de validation	• Toutes les informations sont bien présentes durant le combat • Les informations se mettent à jour en temps réel

4.2. Fonctionnalités du backend

Nom de la fonctionnalité	Route login
Description de la fonctionnalité	Cette permet au joueur de se connecter au jeu en passant par l'API
Comportement de la fonctionnalité	Le joueur donnera ces informations : • Email • Mot de passe Ensuite si le server API vérifie si le compte existe et si c'est le cas, le joueur recevra un token pour qu'il puisse naviguer dans les menus
Contrainte technique	
Critère de validation	• L'application peut faire appel à cette route • Les infos données par le joueur sont au bon format pour la route • Le serveur APi accède bien à la base de données • L'appel de la route renvoie bien une réponse (le compte existe bien ou non en l'occurrence) • Un token est bien généré

Nom de la fonctionnalité	Route authentication
Description de la fonctionnalité	Cette permet au joueur de se créer un compte sur le jeu en passant par l'API
Comportement de la fonctionnalité	Le joueur donnera ces informations : • Nom • Prénom • Nom d'utilisateur • Numéro de téléphone • Date de naissance • Lieu d'habitation • Email • Mot de passe Ensuite le server API crée un compte au joueur et sera connecté avec cette dernière, le joueur recevra un token pour qu'il puisse naviguer dans les menus
Contrainte technique	
Critère de validation	• L'application peut faire appel à cette route • Les infos données par le joueur sont au bon format pour la route • Le serveur API accède bien à la base de données • L'appel de la route renvoie bien une réponse (le compte existe bien ou non en l'occurrence) • Un token est bien généré

Nom de la fonctionnalité	CRUD user
Description de la fonctionnalité	CRUD qui permet d'interagir avec les utilisateurs stockés dans la base de données
Comportement de la fonctionnalité	Pour pouvoir interagir avec les utilisateurs stockés dans la base de données, il faut créer les routes suivantes : • la route get_all • la route get one • la route create • la route update • la route delete chaque route devra donner une réponse en retour

Contrainte technique	
Critère de validation	• Les routes sont créées • Les erreurs sont bien gérés • Les routes renvoie bien la réponse attendu • il faut bien un token pour utiliser les routes

Nom de la fonctionnalité	CRUD Robot
Description de la fonctionnalité	CRUD qui permet d'interagir avec les robots stockés dans la base de données
Comportement de la fonctionnalité	Pour pouvoir interagir avec les robots stockés dans la base de données, il faut créer les routes suivantes : • la route <code>get_all</code> • la route <code>get one</code> • la route <code>get robot by player _id</code> • la route <code>create</code> • la route <code>update</code> • la route <code>delete</code> chaque route devra donner une réponse en retour
Contrainte technique	
Critère de validation	• Les routes sont créées • Les erreurs sont bien gérés • Les routes renvoie bien la réponse attendu • il faut bien un token pour utiliser les routes

Nom de la fonctionnalité	CRUD Game Session
Description de la fonctionnalité	CRUD qui permet d'interagir avec les sessions de jeu dans la base de données
Comportement de la fonctionnalité	Pour pouvoir interagir avec les sessions de jeu stockés dans la base de données, il faut créer les routes suivantes : • la route <code>get_all</code> • la route <code>get one</code> • la route <code>get robot by host_player</code> • la route <code>create</code> • la route <code>update</code>

	la route delete chaque route devra donner une réponse en retour
Contrainte technique	
Critère de validation	- Les routes sont créées - Les erreurs sont bien gérés - Les routes renvoie bien la réponse attendu - il faut bien un token pour utiliser les routes

4.3. Infrastructure cloud basée sur Microsoft Azure

4.3.1. Introduction

Cette partie détaille l'infrastructure cloud possible basée sur Microsoft Azure nécessaire pour le développement et l'exploitation de notre jeu mobile en réalité augmentée (AR) axé sur la collection et le combat de robots personnalisables. L'objectif est de garantir une expérience utilisateur fluide, sécurisée et évolutive pour les fonctionnalités de gestion des joueurs, multijoueur, économie du jeu et distribution des actifs.

4.3.2. Contexte

Effectivement notre jeu repose sur Unity avec AR Foundation pour le frontend et nécessite un backend robuste pour gérer :

- L'authentification et les profils des joueurs.
- Les sessions multijoueurs en temps réel.
- Les transactions en jeu via money virtuelle (achats, remboursements).
- Le stockage et la distribution des actifs (modèles 3D, textures).
- La surveillance des performances et l'analytique.

Azure est choisi pour sa scalabilité, ses services spécifiques au gaming (Azure PlayFab) et son intégration avec Unity.

4.3.3. Services Azure requis

Service Azure	Description	Rôle dans le projet
Azure PlayFab	Plateforme backend pour jeux	Authentification, gestion des données joueurs, matchmaking multijoueur, économie du jeu, leaderboards, analytique
Azure Cosmos DB	Base de données NoSQL évolutive	Stockage des données dynamiques non couvertes par PlayFab (ex. : historique des combats, quêtes personnalisées)
Azure Blob Storage	Stockage d'objets	Hébergement des actifs du jeu (modèles 3D, textures, animations)

Azure CDN	Réseau de distribution de contenu	Distribution rapide des actifs aux joueurs à l'échelle mondiale
Azure Functions	Calcul serverless	Exécution de logiques backend personnalisées (ex. : traitement des remboursements, quêtes spécifiques)
Azure Monitor/Log Analytics	Surveillance des performances	Suivi des performances, diagnostic des erreurs et optimisation
Azure Key Vault	Gestion des secrets	Stockage sécurisé des clés API, mots de passe et autres secrets

4.4. Architecture cloud

4.4.1. Vue d'ensemble

- **Frontend** : Application Unity déployée sur iOS et Android, communiquant avec le backend via des API REST ou SDK PlayFab.
- **Backend** : Azure PlayFab comme cœur du backend, gérant l'authentification, les données joueurs, le multijoueur et l'économie.
- **Stockage** :
 - Azure Cosmos DB pour les données structurées dynamiques (progression, quêtes, statistiques).
 - Azure Blob Storage pour les actifs statiques (modèles, textures).
- **Distribution** : Azure CDN pour réduire la latence lors du téléchargement des actifs.
- **Logique personnalisée** : Azure Functions pour des traitements spécifiques (ex. : validation des transactions).
- **Sécurité** : Azure Key Vault pour gérer les secrets et sécuriser les communications.
- **Surveillance** : Azure Monitor et Log Analytics pour détecter les anomalies et optimiser les performances.

4.4.2. Détails des composants

4.4.2.1. Azure PlayFab

- **Authentification** : Gestion des comptes joueurs (connexion via e-mail, réseaux sociaux, ou anonyme).
- **Données joueurs** : Stockage des profils, collections de robots, statistiques et succès.
- **Multijoueur** : Matchmaking automatique/local, synchronisation des sessions en temps réel via PlayFab Multiplayer Services.
- **Économie** : Gestion de la monnaie virtuelle, achats en boutique (robots, boîtes mystères), remboursements.
- **Leaderboards** : Classements pour encourager la compétition.
- **Analytique** : Suivi des comportements pour optimiser l'engagement et la rétention.

4.4.2.2. Azure Cosmos DB

- Stockage des données non couvertes par PlayFab, comme l'historique détaillé des combats ou les quêtes personnalisées.
- Partitionnement par ID joueur pour une scalabilité optimale.
- API SQL pour une intégration facile avec Unity.

Le choix d'**Azure Cosmos DB** au lieu de **MSSQL** (Azure SQL Database) repose sur les besoins spécifiques de notre jeu mobile AR, qui privilégient la scalabilité, la flexibilité et la performance globale. Voici une comparaison détaillée :

1. Nature des données et structure

- **Cosmos DB** : C'est une base de données NoSQL qui gère des données semi-structurées ou non structurées (JSON). Dans un jeu, les données comme les profils joueurs, les collections de robots, les quêtes ou l'historique des combats varient souvent en structure (ex. : un robot peut avoir des attributs différents selon son type). Cosmos DB s'adapte facilement à ces variations sans nécessiter un schéma rigide.

- **MSSQL** : C'est une base de données relationnelle SQL qui impose un schéma fixe (tables, colonnes). Cela peut compliquer la gestion de données hétérogènes ou évolutives, nécessitant des modifications fréquentes du schéma pour ajouter de nouveaux types de robots ou fonctionnalités.

2. Scalabilité

- **Cosmos DB** : Conçue pour une scalabilité horizontale automatique, elle peut gérer des millions de requêtes par seconde et des volumes massifs de données à l'échelle mondiale. Elle partitionne les données par clé (ex. : ID joueur) pour équilibrer la charge, ce qui est idéal pour un jeu avec une base de joueurs potentiellement mondiale.
- **MSSQL** : Bien qu'évolutive, MSSQL repose sur une scalabilité verticale (augmentation des ressources d'un serveur) ou des techniques comme le sharding, qui sont plus complexes à configurer et moins flexibles pour des charges imprévisibles. Cela peut poser problème lors de pics de connexion (ex. : événements en jeu).

3. Performance et latence

- **Cosmos DB** : Offre une latence garantie à moins de 10 ms pour les lectures/écritures grâce à sa réplication multi-régions et son architecture optimisée. Pour un jeu multijoueur en temps réel, cette faible latence est critique pour synchroniser les données (ex. : progression, scores).
- **MSSQL** : Bien que performant, MSSQL peut avoir des latences plus élevées pour des charges globales, surtout sans une configuration complexe de réplication géographique. Cela peut affecter l'expérience dans les combats multijoueurs.

4. Distribution mondiale

- **Cosmos DB** : Supporte nativement la réplication multi-régions, permettant de stocker les données près des joueurs (ex. : Europe, Asie, Amérique). Cela réduit la latence pour les utilisateurs distants, un facteur clé pour un jeu mobile mondial.
- **MSSQL** : La réplication géographique est possible, mais plus complexe à configurer et coûteuse, nécessitant des instances Azure SQL distinctes par région.

5. Intégration avec PlayFab

- **Cosmos DB** : S'intègre bien avec Azure PlayFab, qui gère déjà une grande partie des données joueurs (profils, inventaires). Cosmos DB complète PlayFab pour des données spécifiques (ex. : historique des combats, quêtes dynamiques) grâce à son API SQL, compatible avec les requêtes Unity.
- **MSSQL** : Bien qu'intégrable, MSSQL nécessite des connecteurs SQL standards, ce qui peut compliquer l'interopérabilité avec PlayFab, qui privilégie des formats JSON/NoSQL.

6. Flexibilité pour les mises à jour

- **Cosmos DB** : Permet d'ajouter de nouvelles propriétés ou types de données sans modifier un schéma, facilitant les mises à jour fréquentes (ex. : nouveaux robots, mécaniques de jeu).
- **MSSQL** : Les modifications de schéma (ex. : ajout de colonnes pour de nouvelles stats de robots) nécessitent des migrations, ce qui peut entraîner des temps d'arrêt ou des erreurs.

7. Coût

- **Cosmos DB** : Bien que potentiellement plus coûteux pour de faibles volumes, son modèle de tarification basé sur le débit (RU/s) est prévisible et adapté aux charges variables d'un jeu. La scalabilité automatique réduit les coûts d'administration.
- **MSSQL** : Peut être moins cher pour des charges stables, mais la gestion manuelle de la scalabilité (ex. : sharding, réplication) augmente les coûts opérationnels à long terme.

8. Cas d'usage du jeu

- Le jeu nécessite un stockage rapide et flexible pour :
 - Les profils joueurs (données variées comme les collections, statistiques).
 - Les quêtes dynamiques (journalières, hebdomadaires, spécifiques).
 - L'historique des combats (données volumineuses pour l'analytique).
- Cosmos DB excelle dans ces scénarios grâce à sa capacité à gérer des données JSON, sa scalabilité et sa faible latence. MSSQL serait plus adapté à des applications avec des relations complexes (ex. : systèmes ERP), ce qui n'est pas le cas ici.

4.4.2.3. Azure Blob Storage

- Stockage des modèles 3D, textures et animations.

- Organisation en conteneurs (ex. : /assets/robots, /assets/textures).
- Politiques d'accès sécurisées via signatures d'accès partagé (SAS).

4.4.2.3. Azure CDN

- Intégré à Blob Storage pour distribuer les actifs avec faible latence.
- Points de présence mondiaux pour une couverture optimale.

Azure CDN (Content Delivery Network) est un réseau de distribution de contenu qui accélère la livraison des actifs numériques (comme les modèles 3D, textures, animations) aux utilisateurs finaux. Voici pourquoi il est essentiel pour notre projet :

- **Réduction de la latence** : Azure CDN utilise un réseau mondial de serveurs (points de présence, ou PoP) pour mettre en cache les actifs près des joueurs. Par exemple, un joueur en Asie télécharge les fichiers depuis un PoP local plutôt que directement depuis un serveur central aux États-Unis, ce qui réduit le temps de chargement.
- **Amélioration de l'expérience utilisateur** : Dans un jeu mobile AR, les joueurs s'attendent à des chargements rapides des modèles de robots et des textures, surtout lors du lancement d'un combat. CDN garantit une distribution fluide, même avec des connexions variables.
- **Scalabilité** : CDN gère les pics de trafic (ex. : lors de mises à jour ou d'événements en jeu) en répartissant la charge sur plusieurs PoP, évitant ainsi la surcharge des serveurs principaux (Azure Blob Storage dans ce cas).
- **Intégration avec Azure Blob Storage** : Les actifs stockés dans Blob Storage sont automatiquement distribués via CDN, simplifiant la configuration.

4.4.2.4. Azure Functions

- Logiques serverless pour :
 - Validation des transactions (ex. : vérification des achats).
 - Gestion des quêtes dynamiques (ex. : attribution de récompenses).
 - Traitement des remboursements.
- Déclencheurs HTTP ou basés sur des événements PlayFab.

4.4.2.5. Azure Monitor/Log Analytics

- Collecte des journaux de PlayFab, Cosmos DB et Functions.
- Tableaux de bord pour suivre la latence, les erreurs et l'utilisation des ressources.
- Alertes pour les anomalies critiques (ex. : pics de latence).

4.4.2.5. Azure Key Vault

- Stockage des clés API PlayFab, secrets Cosmos DB et autres identifiants.
- Intégration avec Azure Functions pour un accès sécurisé.

4.5. Considérations AR

- Le jeu utilise Unity AR Foundation pour les fonctionnalités AR client-side (détection de surfaces, occlusion, suivi de mouvement).
- Azure Spatial Anchors, initialement pertinent pour les ancrs AR partagées, a été retiré le 20 novembre 2024 (Azure Spatial Anchors Retirement). Si des sessions AR partagées sont nécessaires (ex. : robots visibles dans le même espace AR pour plusieurs joueurs), une solution tierce ou personnalisée devra être développée, augmentant potentiellement la complexité.

Explication des considérations AR et des alternatives possibles

4.5.1. Contexte des considérations AR

Le jeu mobile en réalité augmentée (AR) utilise **Unity AR Foundation** pour gérer les fonctionnalités AR côté client (sur les appareils iOS et Android). Ces fonctionnalités incluent :

- **Détection de surfaces** : Identifier les surfaces planes (comme une table ou le sol) pour placer les robots virtuels dans l'environnement réel.
- **Occlusion** : Permettre aux objets virtuels (robots) d'être masqués par des objets réels (ex. : un robot partiellement caché derrière un meuble).
- **Suivi de mouvement** : Suivre les mouvements de l'appareil pour maintenir la cohérence des objets AR dans l'espace réel.

Cependant, notre projet mentionne un besoin potentiel pour des **sessions AR partagées**, où plusieurs joueurs peuvent voir et interagir avec les mêmes objets AR (ex. : deux joueurs voyant le même robot dans le même espace physique lors d'un combat local). Cela nécessite une synchronisation des positions des objets AR entre les appareils, une fonctionnalité complexe appelée **ancrage AR partagé**.

Azure Spatial Anchors, un service cloud d'Azure, était initialement conçu pour répondre à ce besoin en permettant de créer et de partager des ancres AR persistantes entre plusieurs appareils. Cependant, ce service a été **retiré le 20 novembre 2024** ([Azure Spatial Anchors Retirement](#)), obligeant l'équipe à envisager des alternatives si des sessions AR partagées sont nécessaires.

4.5.2. Rôle de Unity AR Foundation

Unity AR Foundation est un framework intégré à Unity qui abstrait les SDK natifs ARKit (iOS) et ARCore (Android), permettant de développer des applications AR multiplateformes avec un seul codebase. Dans ce projet, AR Foundation gère :

- **Rendu des robots** : Placement des modèles 3D dans l'environnement réel.
- **Interactions AR** : Détection des surfaces pour créer l'arène de combat, gestion des obstacles virtuels/réels.
- **Expérience locale** : Chaque joueur voit ses robots et l'arène sur son propre appareil.

Cependant, AR Foundation ne prend pas en charge nativement les **sessions AR partagées** (ex. : synchroniser la position d'un robot entre deux joueurs dans un espace physique partagé). Cela nécessite une solution backend pour partager les coordonnées des ancres AR.

4.5.3. Rôle initial d'Azure Spatial Anchors

Azure Spatial Anchors permettait de :

- Créer des **ancres AR** : Points fixes dans l'espace réel associés à des objets virtuels (ex. : un robot placé sur une table).
- **Persistance** : Sauvegarder ces ancres dans le cloud pour qu'elles restent cohérentes entre sessions.
- **Partage** : Permettre à plusieurs appareils de localiser la même ancre, synchronisant ainsi les objets AR pour une expérience partagée (ex. : deux joueurs voyant le même robot au même endroit).
- **Cas d'usage dans le jeu** : Dans un mode multijoueur local, les joueurs pourraient se réunir physiquement, scanner le même environnement, et voir leurs robots combattre dans une arène AR partagée.

Le retrait de Spatial Anchors signifie que cette fonctionnalité n'est plus disponible directement via Azure, ce qui complique la mise en œuvre des sessions AR partagées.

4.5.5 Impact du retrait d'Azure Spatial Anchors

Sans Spatial Anchors, le jeu peut toujours fonctionner en mode **AR individuel** (chaque joueur voit son propre environnement AR), mais les **sessions AR partagées** nécessitent une alternative. Cela augmente :

- **Complexité** : Développement ou intégration d'une solution personnalisée ou tierce.
 - **Coût** : Ressources supplémentaires pour le développement et la maintenance.
 - **Temps** : Délais prolongés pour tester et valider la solution.
-

4.5.6. Autres choix possibles pour les sessions AR partagées

Voici les alternatives pour remplacer Azure Spatial Anchors et permettre des sessions AR partagées dans le jeu :

a) Solutions tierces

1. Niantic Platform (Lightship ARDK)

- **Description** : La plateforme AR de Niantic (créateurs de Pokémon GO) propose des outils pour des expériences AR multiplateformes, incluant le **Lightship VPS** (Visual Positioning System) pour des ancres partagées et persistantes.
- **Fonctionnalités** :
 - Synchronisation des objets AR entre plusieurs appareils via des ancres basées sur la localisation.
 - Support pour des sessions multijoueurs AR en temps réel.
 - Intégration avec Unity.
- **Avantages** :
 - Solution mature, conçue pour des jeux AR à grande échelle.
 - Supporte des environnements extérieurs et intérieurs.
 - Documentation et communauté active.
- **Inconvénients** :

- Coût potentiel (modèle de tarification basé sur l'usage).
- Dépendance à une plateforme tierce, ce qui peut limiter la personnalisation.
- Nécessite une intégration supplémentaire avec le backend Azure PlayFab.
- **Pertinence pour le projet** : Idéal si le jeu vise des sessions AR partagées dans des environnements variés (ex. : combats dans un parc ou à domicile).

2. Google Cloud Anchors (via ARCore Cloud Anchors)

- **Description** : Service de Google pour partager des ancres AR entre appareils Android et iOS, similaire à Spatial Anchors.
- **Fonctionnalités** :
 - Création d'ancres AR basées sur des scans environnementaux.
 - Synchronisation des ancres via le cloud pour des expériences partagées.
 - Intégration avec ARCore, compatible avec Unity AR Foundation.
- **Avantages** :
 - Gratuit pour un usage limité (limites de quotas à vérifier).
 - Bonne compatibilité avec Android, plateforme clé pour le jeu.
- **Inconvénients** :
 - Moins robuste pour iOS comparé à ARKit.
 - Nécessite une gestion manuelle de la synchronisation avec le backend Azure.
 - Moins de fonctionnalités avancées par rapport à Niantic.
- **Pertinence pour le projet** : Convient pour un prototype ou un mode multijoueur local simple, mais moins évolutif pour des expériences complexes.

3. 8th Wall

- **Description** : Plateforme AR basée sur le web, offrant des outils pour des expériences AR multiplateformes, y compris des fonctionnalités multijoueurs.
- **Fonctionnalités** :
 - Synchronisation des objets AR via un backend cloud.
 - Support pour des sessions AR partagées sans SDK natifs (fonctionne dans les navigateurs).
 - Intégration possible avec Unity via des plugins.
- **Avantages** :
 - Facile à déployer pour des prototypes rapides.
 - Compatible avec une large gamme d'appareils.
- **Inconvénients** :
 - Performances potentiellement inférieures pour des jeux AR complexes par rapport à AR Foundation.
 - Coût élevé pour un usage à grande échelle.
 - Moins adapté pour des intégrations profondes avec Azure PlayFab.
- **Pertinence pour le projet** : Utile si le jeu envisage une version web ou des fonctionnalités AR accessibles sans installation, mais moins prioritaire pour un jeu mobile natif.

b) Solution personnalisée

- **Description** : Développer une solution interne pour gérer les ancres AR partagées en utilisant les capacités d'AR Foundation, combinées à un backend Azure (ex. : Azure PlayFab, Azure Functions, Cosmos DB).
- **Approche** :
 - Utiliser AR Foundation pour détecter et créer des ancres locales sur chaque appareil.
 - Partager les coordonnées des ancres (position, rotation) entre les joueurs via Azure PlayFab Multiplayer Services.
 - Stocker les données d'ancrage dans Azure Cosmos DB pour la persistance.
 - Synchroniser les transformations des objets AR en temps réel à l'aide de WebSockets ou du système de messagerie de PlayFab.
- **Avantages** :
 - Contrôle total sur la solution, permettant une personnalisation poussée.
 - Intégration native avec l'infrastructure Azure existante (PlayFab, Cosmos DB).

- Pas de dépendance à des services tiers.
- **Inconvénients :**
 - Complexité élevée : nécessite une expertise en AR et en synchronisation réseau.
 - Temps de développement important, pouvant retarder le projet.
 - Risque de bugs ou de problèmes de précision (ex. : désalignement des ancrs entre appareils).
- **Pertinence pour le projet :** Recommandée uniquement si les sessions AR partagées sont une fonctionnalité clé et si l'équipe dispose des ressources pour un développement poussé. Sinon, une solution tierce comme Niantic Lightship est préférable.

c) Abandon des sessions AR partagées

- **Description :** Simplifier le jeu en supprimant la nécessité d'ancres AR partagées, en se concentrant sur des sessions AR individuelles ou des combats multijoueurs en ligne non-AR (ex. : arène virtuelle prédéfinie).
- **Approche :**
 - Chaque joueur utilise son propre environnement AR, sans synchronisation des ancrs.
 - Les combats multijoueurs sont gérés via PlayFab, avec une synchronisation des actions (attaques, déplacements) mais sans alignement spatial.
 - Alternative : Proposer un mode multijoueur local basé sur une arène virtuelle (non-AR) pour les combats en proximité.
- **Avantages :**
 - Réduit considérablement la complexité et les coûts.
 - Compatible avec l'infrastructure Azure PlayFab existante.
 - Aucun besoin de solutions tierces ou personnalisées.
- **Inconvénients :**
 - Perte d'une fonctionnalité immersive (joueurs ne partagent pas le même espace AR).
 - Moins d'attrait pour les modes multijoueurs locaux en AR.
- **Pertinence pour le projet :** Solution viable si les sessions AR partagées ne sont pas essentielles ou si le budget/temps est limité. Cela peut être compensé par un mode multijoueur en ligne robuste.

4.5.7. Recommandation pour le projet

Le choix de la solution dépend des priorités du jeu :

- **Si les sessions AR partagées sont critiques** (ex. : combats locaux où les joueurs doivent voir les mêmes robots dans le même espace) :
 - **Niantic Lightship ARDK** est recommandé pour sa maturité et sa capacité à gérer des expériences AR multijoueurs complexes. Il faudra cependant budgétiser les coûts et planifier l'intégration avec Azure PlayFab.
 - **Solution personnalisée** est une option si l'équipe a les ressources et le temps nécessaires, mais elle est risquée sans expertise approfondie.
- **Si les sessions AR partagées sont secondaires :**
 - **Abandonner cette fonctionnalité** est la solution la plus simple, en se concentrant sur des combats AR individuels et des modes multijoueurs en ligne via PlayFab. Cela réduit les coûts et accélère le développement tout en maintenant une expérience engageante.

-
- [Azure Spatial Anchors Retirement](#)
 - [Niantic Lightship ARDK](#)
 - [Google Cloud Anchors](#)
 - [8th Wall Documentation](#)

5. Gestion des risques

5.1. Risques techniques

- **Latence multijoueur :** La synchronisation en temps réel peut être affectée par une mauvaise configuration de PlayFab.
- **Scalabilité :** Une mauvaise gestion des partitions dans Cosmos DB peut limiter la performance.
- **Distribution des actifs :** Une configuration incorrecte de CDN peut entraîner des temps de chargement élevés.

5.2. Stratégies d'atténuation

- Tests de charge pour PlayFab Multiplayer Services avant le lancement.
- Optimisation des partitions Cosmos DB dès la phase de prototypage.
- Configuration rigoureuse de CDN avec tests régionaux.
- Surveillance proactive via Azure Monitor pour détecter et résoudre les problèmes rapidement.

6. Ressources nécessaires

Rôle	Compétences requises	Tâches
Développeur Backend	Azure PlayFab, Azure Functions, Cosmos DB	Configuration et maintenance des services cloud
Administrateur Cloud	Azure Monitor, Key Vault, CDN	Surveillance, sécurité et optimisation
Testeur QA	Tests de performance, tests de charge	Validation de la scalabilité et de la latence

L'infrastructure Azure proposée, centrée sur PlayFab, Cosmos DB, Blob Storage et CDN, répond aux besoins du jeu en termes de scalabilité, performance et sécurité. Une attention particulière devra être portée à la gestion de la latence multijoueur et à l'exploration de solutions alternatives pour les fonctionnalités AR partagées, en raison du retrait d'Azure Spatial Anchors.